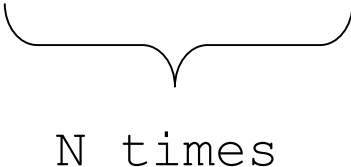


Loop structures

Djaafri

Introduction (1)

- If we want to calculate the sum of $7 + 7$, the result is 14.
 $s \leftarrow 7 + 7;$ (repeat the addition 1 time)
- If we want to calculate the sum of $7 + 7 + 7$:
 $s \leftarrow 7 + 7 + 7;$ (repeat the addition 2 times)
- If we want to calculate the sum of $7 + 7 + \dots + 7$:

N times

How to repeat an instruction N times in an algorithm?

Introduction (2)

- One may hope that the following solution works:

$a \leftarrow 7 + 7 + \dots + 7; \text{ (N additions)}$

- Problem: the « ... » is not an allowed operator in algorithmic language.

Introduction(3)

- Algorithmic language allows expressing repetition.

A repetition structure allows executing an instruction or a group of instructions multiple times.

The instructions remain the same, but the processed data changes with each repetition. This is called a repetition structure, iteration structure, or loop.

Type de structure de boucles

- Three types of loops:
 - The '**For**' loop
 - The '**While**' loop
 - The '**Repeat**' loop

The for loop

The FOR loop allows repeating the execution of a set of actions multiple times. It is presented in the following form:

```
for <cpt> ← <vi> to <vf> do
    Action 1 ;
    Action 2 ;
    ...
    Action n ;
end;
```

Instructions repeated N times such that:

$$N = vf - vi + 1$$

- **cpt**: variable that counts the number of iterations
- **vi**: initial value of the counter
- **vf**: endal value of the counter
- The execution of actions is performed **N** times such that: $N = vf - vi + 1$

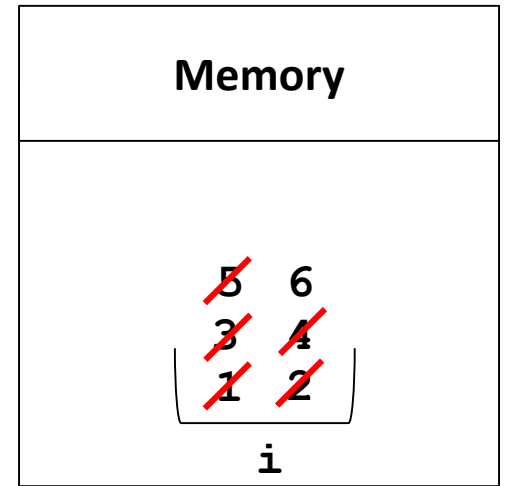
Note:

- ❖ Before the first iteration, **cpt** is asifgned the value **vi**.
- ❖ At the beginning of each iteration, **cpt** is compared with **vf**. If $cpt > vf$, the loop exits.
- ❖ After each iteration (after executing all actions), the **for** loop increments **cpt** ($cpt \leftarrow cpt + 1$).
- ❖ The variable **cpt** is used inifde the **for** loop in read-only mode (it can be used but not modified).
- ❖ The **for** loop is used when the number of iterations is known in advance.

Example1

- write an Algorithm that displays «Hello » 5 times.
- solution:

```
Algorithm Hello5Times;  
var  
  i: integer;  
begin  
  for i ← 1 to 5 do  
    write(" Hello " );  
  end;  
end.
```



Affichage
Hello
Hello
Hello
Hello
Hello

The for loop in C

*Initialization
instruction*

*Continuity
condition*

*Repeated instruction
after each loop*

```
for( <inst_ini> ; <cond_cont> ; <inst_repet> ) {  
  
    Action 1;  
    Action 2;  
    ...  
    Action n;  
  
}
```

*Instructions repeated until
cond_cont becomes false*

inst_ini : initialisation instruction executed before the 1st iteration

cond_cont : continuity condition, evaluated **before each iteration**. If it is true, the body of the for is executed, otherwise the loop is exited.

inst_repet : instruction repeated **after each iteration**

Example1 en C

```
#include <stdio.h>
int main() {
    int i ;
    for(i=1; i<=5; i=i+1) {
        printf("Hello\n");
    }
    return 0;
}
```

Example2

- How many “Hello” messages will the following algorithm display?

```
Algorithm Example2;  
var  
    i: integer;  
begin  
    for i ← 15 to 32 do  
        write(" Hello " );  
    end;  
end.
```

- Answer: number of “Hello” displayed is:

$$\text{nb}=32-15+1=18$$

Example 3: multiplying a by b by succesifve addition: **for loop**

- Assume that the * (multiplication) operator does not exist
- Replace the * operator with several additions

Example :

$$7 \times 5 = 7 + 7 + 7 + 7 + 7$$


5 times

Example 3: multiplying a by b by succesifve addition: **for loop**

Analysifs

a=7

b=5

i=1 S = 0 + 7 = 7

i=2 S = 7 + 7 = 14

i=3 S = 14 + 7 = 21

i=4 S = 21 + 7 = 28

i=5 S = 28 + 7 = 35

i=6 stop because i > b (b=5)

The calculation was completed in 5 iterations, and in each iteration we have :

$s \leftarrow s + 7;$

Algorithm

```
Algorithm multipParAdd;  
var  
    a, b, s, i: integer;  
begin  
    write("Enter a and b:");  
    read(a); read(b);  
    s ← 0;  
  
    for i ← 1 to b do  
        s ← s + a;  
    end;  
  
    write(a, "x" , b, "=", s);  
end.
```

Example 3: multiplying a by b by successive addition: C for loop

```
#include <stdio.h>

int main() {
    int i , a , b , s ;
    printf(" donner a et b \n");
    scanf("%d",&a);
    scanf("%d",&b);
    s = 0 ;
    for(i=1; i<=b; i=i+1) {
        s = s + a ;
    }
    printf("%d x %d = %d", a , b , s);
    return 0;
}
```

Example 4: sum of all integers between two values *a* and *b*

Analysifs

a=8

b=12

i=8 S = 0 + 8 = 8

i=9 S = 8 + 9 = 17

i=10 S = 17 + 10 = 27

i=11 S = 27 + 11 = 38

i=12 S = 38 + 12 = 50

i=13 stop because i > b (b=12)

The calculation was completed in
12-8+1 = 5 iterations, and in each
iteration we have :

s ← s + i;

Algorithm

Algorithm somme;

var

a, b, s, i: integer;

begin

write("Donner a et b :");

read(a,b);

s ← 0;

for i ← a **to** b **do**

s ← s + i;

end;

write(" la somme des nombres
entre" , a, " et ", b, " = " , s);

end.

Example 4: sum of all integers between two values *a* and *b* (*version 1*)

```
#include <stdio.h>
```

```
int main() {
```

```
    int i , a , b , s ;
```

```
    printf(" Give two integer a and b \n");
```

```
    scanf("%d",&a);
```

```
    scanf("%d",&b);
```

```
    s = 0 ;
```

```
    for(i= a; i<=b; i=i+1) {
```

```
        s = s + i ;
```

```
    }
```

```
    printf(" the sum of numbers betwwen %d and %d = %d ", a , b ,  
    s);
```

```
    return 0;
```

```
}
```

- *i* begin by *a* (the smallest value)

- increments in each iteration

- condition of continuity : *i* <= *b*

example 4: sum of all integers between two values *a* and *b* (*verifon 2*)

```
#include <stdio.h>

int main() {
    int i , a , b , s ;
    printf(" donner deux nombres integer a et b \n");
    scanf("%d",&a);
    scanf("%d",&a);
    s = 0 ;
    for(i= b; i>=a; i=i-1) {
        s = s + i ;
    }
    printf(" la somme des nombres entre %d et %d = %d ", a , b , s);
    return 0;
}
```



- *i* starts at *b* (the largest)
- Decrements in each iteration
- Continuity condition: *i* >= *a*

La boucle while:

The while loop is used to repeat the execution of a set of actions when a given condition is met:

```
<initialisation>
while <continuity_condition> do
    Action 1 ;
    Action 2 ;
    ...
    Action n ;
    <Action_on_condition;>
end;
```

Note:

- if **<condition_continuity>** is false before the first iteration then the actions inside while are not executed.
- if the value of **<condition_continuity>** does not change inside the loop and remains equal to true, then the while loop is an endless loop and the Algorithm executes without stopping.
- **<initialisation>** : instructions that give the first value to the while condition
- **<continuity_condition>**: this condition is checked in each iteration, we continue to loop if it is equal to **true**, we exit when it is equal to **false**.
- **<Action_on_condition>**: instructions for setting the continuity condition to **false**

La boucle **while** en C: **while**

*Continuity
Condition*

```
while(<cond_cont>) {
```

```
    Action 1;  
    Action 2;  
    ...  
    Action n;
```

```
}
```

*Body of the while :
Instructions repeated until
cond_cont becomes false*

cond_cont : continuity condition, evaluated **before each iteration**. If it is **true**, the body of the while is executed, else the loop is exited.

example2 : while

- what this algorithm does ?

```
Algorithm example2;  
var  
    i: integer;  
begin  
    i ← 15;  
    while i ≤ 32 do  
        write(" Hello " );  
        i ← i + 1;  
    end;  
end.
```

- **Response:** it loops endlessly and displays "Hello" non-stop.
 $nb = 32 - 15 + 1 = 18$

Example5: Sum of natural numbers less than or equal to n: while loop

Algorithm with for

```
Algorithm SommeNNombres;  
var  
  n, s, i: integer;  
begin  
  write("Enter N :");  
  read(n);  
  s ← 0;  
  for i ← 1 to n do  
    s ← s + i;  
  end;  
  
  write(" Sum=" , s);  
  
end.
```

Algorithm with while

```
Algorithm SommeNNombres;  
var  
  n , s, i: integer;  
begin  
  write("Enter N:");  
  read(n);  
  s ← 0;  
  i ← 1; Initialisation  
  while i ≤ n do Condition de continuité  
    s ← s + i;  
    i ← i + 1; Action sur la condition  
  end;  
  
  write(" Sum=" , s);  
  
end.
```

Example 5: Multiplication by successive addition: **while loop**

Algorithm with **for**

```
Algorithm produitParSomme;  
var  
  a, b, s, i: integer;  
begin  
  write(" Enter a and b:");  
  read(a); read(b);  
  s ← 0;  
  for i ← 1 to b do  
    s ← s + a;  
  end;  
  
  write(a, "x" , b, "=", s);  
  
end.
```

Algorithm with **while**

```
Algorithm produitParSomme;  
var  
  a, b , s, i: integer;  
begin  
  write(" Enter a and b:");  
  read(a); read(b);  
  s ← 0;  
  i ← 1;  
  while i ≤ b do  
    s ← s + a;  
    i ← i + 1;  
  end;  
  
  write(a, "x" , b, "=", s);  
  
end.
```

Initialisation

Continuity Condition

Action on condition

La boucle repeat

The **repeat loop** allows you to repeat the execution of a set of actions **until** a **given condition** becomes **true**. It is structured as follows:

Repeat

```
Action 1 ;
```

```
Action 2 ;
```

```
...
```

```
Action n ;
```

```
<Action_on_condition;>
```

```
until <stoppin_condition>;
```

<Stopping_condition>: This condition is checked at the end of each iteration; the loop exits when it becomes **true**.

<Actions_on_condition>: instructions to make the stopping condition **true**

Note: Unlike the **while** loop, the **repeat** loop executes **at least one iteration**.

The Repeat loop in C: do..while

```
do {
```

```
    Action 1;  
    Action 2;  
    ...  
    Action n;
```

Body of the loop :Instructions repeated until `cond_cont` becomes false

```
} while (<cond_cont>) ;
```

Continuity Condition

cond_cont : continuity condition, evaluated **after each iteration**, if **true**, execute the body of the while, else exit the loop

Note: unlike the **while loop**, **do .. while** allows you to perform at least one iteration.

Example 5: multiplication by successive addition : **repeat** loop

Algorithm with while

```
Algorithm MultiBySum;
var
  a, b , s, i: integer;
begin
  write("Enter a and b:");
  read(a); read(b);
  S ← 0;
  i ← 1;

  while i ≤ b do
    s ← s + a;
    i ← i + 1;
  end;

  write(a, "x" , b, "=", s);

end.
```

Algorithm with repeat

```
Algorithm MultiBySum;
var
  a, b , s, i: integer;
begin
  write("Enter a and b:");
  read(a); read(b);
  S ← 0;
  i ← 1;

  repeat
    s ← s + a;
    i ← i + 1;
  until i > b;

  write(a, "x" , b, "=", s);

end.
```

Exercise 1: Even or odd by subtraction

- Determine whether a given positive integer is even or odd using successive subtraction. **Solution :**

Analysis

a=7

$$\begin{aligned} b=7 \quad b &= 7 - 2 = 5 \\ b &= 5 - 2 = 3 \\ b &= 3 - 2 = 1 \end{aligned}$$

We stop because $b < 2$

b=1 then **7 is odd**

In each iteration, we have:

$$b \leftarrow b - 2;$$

a=8

$$\begin{aligned} b=8 \quad b &= 8 - 2 = 6 \\ b &= 6 - 2 = 4 \\ b &= 4 - 2 = 2 \\ b &= 2 - 2 = 0 \end{aligned}$$

We stop because $b < 2$

b=0 then **8 is even**

In each iteration, we have:

$$b \leftarrow b - 2;$$

Exercise1: Even or odd by subtraction

Algorithm

```
Algorithm nombrePair;
var
    a ,b      : integer;
begin
    write("Enter an integer :"); read(a);
    b ← a;
    while b >= 2 do
        b ← b - 2;
    end;
    if b =0 then
        write(a, " est pair:");
    else
        write(a, " est impair:");
    end;
end.
```

Exercise1: Even or odd by subtraction C Program

```
#include <stdio.h>
int main() {
    int a,b;
    printf("Enter an integer value : ");
    scanf("%d",&a);
    b=a;
    while (b>=2){
        b = b-2;
    }
    if(b==1) printf("%d is odd",a);
    else printf("%d is even",a);
    return 0;
}
```

Exercise 2 : the product of all numbers between two integers a et b (**for loop** version)

a=6

b=9

Initialisation: $i = 6$
 $p = 1$

$i=6$ $p = 1 * 6 = 6$

$i=7$ $p = 6 * 7 = 42$

$i=8$ $p = 42 * 8 = 366$

$i=9$ $p = 366 * 9 = 3024$

$i=10$ $i > 9$ then stop

In each iteration, we have :

$p \leftarrow p * i;$

Algorithm Exercise3_for;

var

a, b, p, i : integer;

begin

write("Enter a and b:");

read(a); read(b);

$p \leftarrow 1;$

for $i \leftarrow a$ to b **do**

$p \leftarrow p * i;$

end;

write(" the product of all
numbers between :", a, " et
", b, " = ", p);

end.

Exercise 2: the product of all numbers between two integers a and b (while loop version 1)

```
Algorithm Exercise3_for;  
var  
  a , b , p , i: integer;  
begin  
  write("Enter a and b:");  
  read(a); read(b);  
  p ← 1;  
  for i ← a to b do  
    p ← p * i;  
  end;  
  write(" the product of all  
numbers between :", a, " et  
", b, " = ", p);  
end.
```

```
Algorithm Exercise3_while;  
var  
  a , b , p , i: integer;  
begin  
  write("Enter a and b:");  
  read(a); read(b);  
  p ← 1;  
  i ← a;  
  while i ≤ b do  
    p ← p * i;  
    i ← i + 1;  
  end;  
  write(" the product of all  
numbers between :", a, " et ",  
b, " = " , p);  
end.
```

Exercise 2: the product of all numbers between two integers a and b (**while loop version 2**)

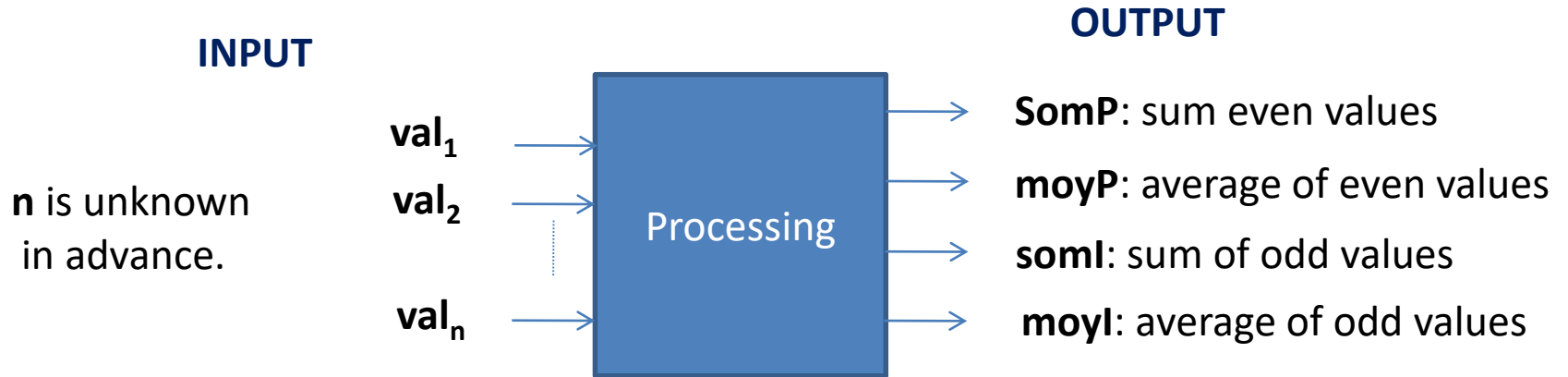
```
Algorithm Exercise3_While1;
var
  a , b , p , i: integer;
begin
  write("Enter a and b:");
  read(a); read(b);
  p ← 1;
  i ← a;
  while i ≤ b do
    p ← p * i;
    i ← i + 1;
  end;
  write( " the product of all
numbers between :", a, " et ",
b, " = " ", p);
end.
```

```
Algorithm Exercise3_While2;
var
  a , b , p , i: integer;
begin
  write("Enter a and b:");
  read(a); read(b);
  p ← 1;
  i ← b;
  while i ≥ a do
    p ← p * i;
    i ← i - 1;
  end;
  write( " the product of all
numbers between :", a, " et ",
b, " = " ", p);
end.
```

Exercise 3

- Write an algorithm and a program that:
 - Reads a set of integer values. Stops reading when the value = 0.
 - Calculates the sum and average of even numbers.
 - Calculates the sum and average of odd numbers.

Exercise 3 - Analyze



- The number of values to read is unknown in advance, so we use a **while** loop.
- Check for each number whether it is a multiple of 2 or not.
- Use a variable to sum the even numbers and another variable to count them.
- Use a variable to sum the odd numbers and another variable to count them.
- Calculate the average of the even numbers and the average of the odd numbers.

Exercise 3: example d'exécution

somP =0 somI =0 cptP =0 cptI =0

read val **val=6** val is even:

somP = 0+6 = 6

cptP = 0+1 = 1

read val **val=7** val is odd :

somI = 0+7 = 7

cptI = 0+1 = 1

read val **val=4** val is even :

somP = 6+4 = 10

cptP = 1+1 = 2

read val **val=2** val is even :

somP = 10+2 = 12

cptP = 2+1 = 3

read val **val=3** val is odd :

somI = 3+7 = 10

cptI = 1+1 = 2

read val **val=0** Stop the repetition

moyP =12/3 = 4.0 moyI =10/2 = 5.0

Mémoire

0

~~3~~

~~2~~

~~4~~

~~7~~

~~6~~

val

12

3

~~10~~

~~2~~

10

2

~~6~~

~~1~~

~~7~~

~~1~~

~~0~~

~~0~~

~~0~~

~~0~~

somP

cptP

somI

cptI

4.0

5.0

moyP

moyI

Exercise 3 – the Algorithm

```
Algorithm Exercise4;
var
    val , somP , somI , cptP , cptI : integer;
    moyP , moyI                      : reel;
begin
    write("Enter a value:");
    read(val);
    somP ← 0; somI ← 0;
    cptP ← 0; cptI ← 0;
    while val <> 0 do
        if val mod 2 = 0 then
            somP ← somP + val;
            cptP ← cptP + 1;
        else
            somI ← somI + val;
            cptI ← cptI + 1;
        end;
        write("Enter another value:");
        read(val);
    end;
    moyP ← somP/cptP;
    moyI ← somI/cptI;
    write("The average of even numbers :", moyP);
    write("The average of odd numbers :", moyI);
end.
```

cptP and **cptI** can be null
⇒ Check their values
before calculating the
averages

Exercise 3 - l'Algorithm amélioré

```
Algorithm Exercise4;
var
  val , somP , somI , cptP , cptI : integer;
  moyP , moyI : reel;
begin
  write("Enter a value:");
  read(val);
  somP ← 0; somI ← 0;
  cptP ← 0; cptI ← 0;
  while val <> 0 do
    if val mod 2 = 0 then
      somP ← somP + val;
      cptP ← cptP + 1;
    else
      somI ← somI + val;
      cptI ← cptI + 1;
    end;
    write("Enter another value:");
    read(val);
  end;
  if cptP <> 0 then
    moyP ← somP/cptP;
    write("The average of even numbers" , moyP);
  end;
  if cptI <> 0 then
    moyI ← somI/cptI;
    write("The average of odd numbers" , moyI);
  end;
end.
```

Exercise 3 - programme C

```
int main(){
    int val, somP , somI , cptP , cptI ;
    float moyP , moyI ;
    printf("Enter an integer:");
    scanf("%d",&val);
    somP =0; somI = 0;
    cptP = 0; cptI = 0;
    while(val!= 0) {
        if( val % 2 == 0) {
            somP = somP + val;
            cptP = cptP + 1;
        }
        else{
            somI = somI + val;
            cptI = cptI+ 1;
        }
        printf("Enter an integer:");
        scanf("%d",&val);
    }
    if (cptP != 0) {
        moyP = (float)somP/cptP;
        printf("The average of even numbers is: %g\n" ,moyP);
    }
    if (cptI != 0) {
        moyI = (float)somI/cptI;
        printf("The average of odd numbers:%g" ,moyI);
    }
    return 0;
}
```

Exercise 4 (Nested Loops)

- write an Algorithm and a program that draws a rectangle  consisting of n lines, each containing m stars (*).

example : for $n=7$ et $m=20$

```
*****  
*****  
*****  
*****  
*****  
*****  
*****
```

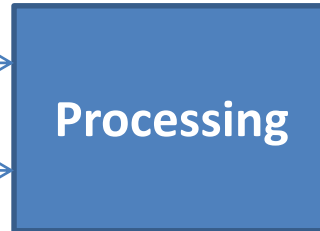
Exercise 4 - Analyze

INPUT

n



m



OUTPUT

Display:

(nxm *) Rectangle

- read the values of n and m
- perform a 1st loop (n times) on the number of lines in the rectangle
- for each line of the rectangle, perform a 2nd loop (m times), for writing the * .

Exercise 4 - Algorithm

```
Algorithm Exercise4;
var
  i , j , n , m : integer;

begin
  write("Give the number of lines in the rectangle:");
  read(n);
  write("Give the number of columns in the rectangle:");
  read(m);
  for i ← 1 to n do
    for j ← 1 to m do
      write ("*");
    end;
    // saut de ligne: C: printf("\n");
  end;
end.
```

Exercise 5

- write an Algorithm and a program that allows you to describe a right-angled triangle (مثلث قائم) made up of n lines and n stars (*) per line.
- **example** : for $n=8$, the program displays:

```
*
* *
* * *
* * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Exercise 5 - analyse

n=8

J =	1	2	3	4	5	6	7	8	9
-----	---	---	---	---	---	---	---	---	---

i=1	*								
-----	---	--	--	--	--	--	--	--	--

i=2	*	*							
-----	---	---	--	--	--	--	--	--	--

i=3	*	*	*						
-----	---	---	---	--	--	--	--	--	--

i=4	*	*	*	*					
-----	---	---	---	---	--	--	--	--	--

i=5	*	*	*	*	*				
-----	---	---	---	---	---	--	--	--	--

i=6	*	*	*	*	*	*			
-----	---	---	---	---	---	---	--	--	--

i=7	*	*	*	*	*	*	*		
-----	---	---	---	---	---	---	---	--	--

i=8	*	*	*	*	*	*	*	*	
-----	---	---	---	---	---	---	---	---	--

i=9									
-----	--	--	--	--	--	--	--	--	--

we have two nested loops;

J=2 > 1 so, stop

J=3 > 2 so, stop

J=4 > 3 so, stop

J=5 > 4 so, stop

J=6 > 5 so, stop

J=7 > 6 so, stop

J=8 > 7 so, stop

J=9 > 8 so, stop

Exercise 5 – Algorithm

```
Algorithm Exercise5;  
var  
  i , j , n : integer;  
  
begin  
  write("Enter n:");  
  read(n);  
  for i ← 1 to n do  
    for j ← 1 to i do  
      write ("*");  
    end;  
  end;  
end.
```

Exercise 5 – Programme C

```
#include <stdio.h>

int main(){
    int i,j,n ;
    printf("Enter n:");
    scanf("%d",&n);
    for(i=1; i <= n; i++) {
        for(j=1; j <= i; j++) {
            printf("*");
        }
        printf("\n"); // return to a new line
    }
    return 0;
}
```

Exercise 6 – home work

- write an algorithm and a program to define a shape made up of n rows and n columns, as shown in the following examples:

→ for $n=7$

```
* * * * *
  * * * *
    * * *
      *
      *
    * * *
  * * * * *
* * * * * *
```

→ for $n=8$

```
* * * * *
  * * * *
    * * *
      * *
      * *
    * * *
  * * * *
* * * * * *
```

Exercise 7

- write a program to draw a shape made up of several lines. The line is made up of n stars (*) if n is even and n plus(+) if n is odd, with $n \geq 0$. Repeat the line display until $n=0$, execution example:

[illegible]

Exercise 7 – Algorithm

```
Algorithm Exercise7;
var
    j , n : integer;
begin
    repeat
        read(n);
    until n >= 0;

    while n <> 0 do
        if n mod 2 = 0 then
            for j ← 1 to n do
                write ("*");
            end;

        else
            for j ← 1 to n do
                write ("+");
            end;
        end;

        repeat
            read(n);
        until n >= 0;
    end;
end.
```

Exercise 7 – Algorithm _amélioré

```
Algorithm Exercise7;
var
  j , n : integer;
  c : character;
begin

  repeat
    repeat read(n); until n>= 0;

    if n mod 2 = 0 then c← '*';
    else                c← '+'; end;

    for j← 1 to n do
      write (c);
    end;

  until n= 0;
end.
```

Exercise 7 – programme C

```
#include <stdio.h>
int main() {
    printf("\tregles d'affichage:\n");
    printf("\t==> n<0 aucun affichage\n");
    printf("\t==> n pair , affichage de n *\n");
    printf("\t==> n impair , affichage de n +\n");
    printf("\t==> n=0 end du programme\n");

    int j,n; char car;
    do{
        do{ scanf("%d",&n); } while(n<0);

        if (n%2==0) // n est pair
            car = '*';
        else // n est impair
            car = '+';
        printf("\t"); // affichage d'une tabulation au début de chaque ligne

        for( j=1;j<=n;j++) printf("%c ",car);

        printf("\n"); // afficher un retour à la ligne
    } while(n!=0);
    return 0;
}
```

Exercise 8

- write a program that calculates the following expression as a function of x and n :

$$s = \frac{1}{x} + \frac{2}{x^3} + \frac{3}{x^5} + \frac{4}{x^7} + \dots + \frac{n}{x^{2n-1}}$$

Exercise 8 – Algorithm

$$s = \frac{1}{x} + \frac{2}{x^3} + \frac{3}{x^5} + \frac{4}{x^7} + \cdots + \frac{n}{x^{2n-1}}$$

```
Algorithm Exercise8;
var
  i , x , n : integer;
  d , s      : real;
begin
  read(n); read(x);
  i ← 1; s ← 0; d ← x;
  while i ≤ n do
    s ← s + i/d;
    d ← d*x*x;
    i ← i + 1;
  end;
end.
```

Note

The number of iterations is known in advance, so you can use the **for loop** instead of the **while loop**.

Exercise 8 – programme C (vers.1)

```
#include <stdio.h>
int main() {
    int i,n; double s, x, d;
    printf("donner un integer n: ");
    scanf("%d",&n);
    printf("donner un real x: ");
    scanf("%lf",&x);
    s=0; i=1; d=x;
    while(i<=n) {
        s=s+i/d;
        d=d*x*x;
        i=i+1;
    }
    printf("s=%lf",s);
    return 0;
}
```

Exercise 8 – programme C (vers.2)

$$s = \frac{1}{x} + \frac{2}{x^3} + \frac{3}{x^5} + \frac{4}{x^7} + \dots + \frac{n}{x^{2n-1}}$$

```
#include <stdio.h>
#include <math.h>
int main() {
    int i,n;
    double s, x;
    printf("donner n: ");
    scanf("%d",&n);
    printf("donner x: ");
    scanf("%lf",&x);
    s=0;
    for(i=1; i<=n; i=i+1){
        s=s+i/pow(x,2*i-1);
    }
    printf("s=%lf\n",s);
    return 0;
}
```